Documentacion tecnica

CV LaTeX Builder

Resumen del documento

Version: v0.9.0
Fecha: 2026-06-11
Estado: Release estable local
Proyecto: documentacion HTML/PDF servida localmente.

Índice

Resumen del proyecto . . . 2
Objetivo del MVP . . . 2
Arquitectura general . . . 2
Modulos implementados . . . 3
- CV Builder Core . . . 3
- Cartas de presentacion . . . 3
- Postulaciones . . . 3
- ATS Basic Check . . . 3
- Editor estructurado de CV (Feature 1 sin publicar) . . . 4
- UI privada y dashboard operativo . . . 4

Docker Compose . . . 4
SQLite y persistencia . . . 4
Exports TEX, PDF y JSON . . . 5
Cartas de presentacion . . . 5
Postulaciones . . . 5
ATS Basic Check . . . 5
Feature 1 sin publicar: editor estructurado, JSON schema 2 y render normalizado . . . 5
- Editor estructurado por secciones . . . 5
- Export/import JSON schema 2 con compatibilidad schema 1 . . . 6
- TEX/PDF y ATS sobre representacion normalizada . . . 6

Git Flow usado . . . 6
Trazabilidad de prompts y comandos . . . 7
PRs y Codex Review . . . 7
Validaciones realizadas . . . 7
Estado release v0.9.0 . . . 8
Riesgos conocidos . . . 8
Backlog posterior . . . 9
Historial y rollback . . . 9

Resumen del proyecto

CV LaTeX Builder es una aplicacion web local para gestionar CVs, cartas de presentacion, postulaciones laborales y un chequeo ATS basico sin servicios externos. El MVP fue construido para ejecutarse con Docker Compose, persistir datos en SQLite y generar artefactos TEX, PDF y JSON de forma controlada.

Objetivo operativo

Mantener una base local estable, portable y auditable para usar datos reales de busqueda laboral sin depender de cloud, IA ni servicios externos.

Objetivo del MVP

- Centralizar la gestion documental de una busqueda laboral en una sola web local.
- Mantener una base portable y reproducible, sin depender de cloud ni autenticacion.
- Validar un flujo completo de CRUD, exportaciones, importaciones y analisis ATS basico.

Arquitectura general

Area	Implementacion
Backend	FastAPI con rutas modulares
Render web	Jinja2 con sidebar fija, dashboard operativo, HTML server-side y CSS con dark/light mode
Persistencia	SQLite en `/data/app.db`
Exportaciones	Servicios Python para TEX, PDF y JSON
PDF	`pdflatex` dentro del contenedor
Assets estaticos	`app/static` servido por FastAPI

Validacion fuerte

El estado estable `v0.9.0` fue validado con Docker Compose, healthcheck HTTP, rutas de documentacion, descarga real de PDFs, extraccion `pdftotext` y suite pytest completa.

Modulos implementados

CV Builder Core

- CRUD completo de CVs.
- Duplicado con validacion de titulo.
- Eliminacion logica con confirmacion.
- Exportaciones TEX, PDF y JSON.
- Importacion JSON con limite de tamano.

Cartas de presentacion

- CRUD basico de cartas.
- Asociacion opcional a un CV existente.
- Exportaciones TEX y PDF.

Postulaciones

- CRUD basico de postulaciones.
- Asociacion opcional a CV y carta.
- Estados: pendiente, enviado, entrevista, rechazado, oferta y pausado.

ATS Basic Check

- Checklist deterministico sobre CVs guardados.
- Score simple, advertencias y recomendaciones.
- Penalizacion de secciones criticas faltantes.

Editor estructurado de CV (Feature 1 sin publicar)

- Editor por secciones con items: datos personales y contacto, titulo profesional, resumen, experiencia, educacion, skills, certificaciones, idiomas, proyectos, links e informacion adicional.
- Alta, edicion, eliminacion y reordenamiento de items con botones `Agregar`, `Quitar`, `Subir` y `Bajar`.
- Validacion estricta en servidor; envios fallidos (422) conservan lo ingresado.
- Estado: implementado en `feature/structured-cv-complete`; todavia sin release (ver Feature 1 sin publicar mas abajo).

UI privada y dashboard operativo

- Sidebar fija para navegar entre Dashboard, CVs, Cartas, Postulaciones, ATS y Documentacion.
- Toggle dark/light persistido en `localStorage`.
- Dashboard privado ya integrado en la linea publicada como `v0.9.0`.
- Dashboard enfocado en CVs y cartas como modulos principales, con `Curriculum Vitae + ATS` como flujo principal.
- Listados compactos con preview visual CSS de documento, badge ATS por CV y acciones secundarias.
- Modal ATS compacto disponible desde el listado de CVs y desde el detalle completo.
- Eliminacion segura de CVs y cartas exigiendo coincidencia exacta del texto mostrado.
- Correccion aplicada para evitar XSS en confirmaciones, renderizando textos dinamicos sin `innerHTML`.

Docker Compose

- Servicio principal `app` con FastAPI.
- Publicacion local por defecto en `127.0.0.1:8000`.
- Healthcheck HTTP sobre `/health`.
- Reinicio `unless-stopped`.

SQLite y persistencia

- La base se inicializa en `/data/app.db`.
- El host monta `./data:/data`.
- Las tablas activas son `cvs`, `cover_letters` y `applications`.
- Se usa `deleted_at` para eliminacion logica en entidades funcionales.

Exports TEX, PDF y JSON

- CVs: TEX, PDF y JSON.
- Cartas: TEX y PDF.
- Los exports persistidos quedan en `/data/exports`.
- Los nombres de archivo se sanitizan y se acotan a `180` caracteres.
- La compilacion PDF ocurre en un temporal controlado.

Cartas de presentacion

El modulo reutiliza la misma infraestructura LaTeX/PDF del modulo CVs, con su propia plantilla `classic_letter.tex` y validaciones especificas para empresa, puesto, saludo, cuerpo y firma.

Postulaciones

El tracker de postulaciones mantiene una asociacion opcional a CV y cover letter, sin integraciones externas ni calendario. Es un modulo local de seguimiento operativo.

ATS Basic Check

El servicio ATS analiza presencia de email, telefono, resumen, experiencia, educacion y skills, ademas de una longitud aproximada del CV. Ningun CV con secciones core faltantes puede quedar en estado `Bueno`.

Feature 1 sin publicar: editor estructurado, JSON schema 2 y render normalizado

El editor estructurado y el schema JSON 2 estan implementados en la rama `feature/structured-cv-complete` y todavia no tienen release: no forman parte de `v0.9.0` ni de ningun tag publicado.

Editor estructurado por secciones

- Secciones soportadas: datos personales y contacto, titulo profesional, resumen, experiencia, educacion, skills, certificaciones, idiomas, proyectos, links e informacion adicional.
- Items con alta, quitar y reordenamiento simple `Subir`/`Bajar` con renumeracion densa automatica; sin drag and drop.
- La validacion es 100 % en servidor: envios invalidos, incompletos o fuera de limites se rechazan con errores accionables en espanol y sin escrituras parciales.
- Un envio fallido (HTTP 422) re-renderiza el formulario conservando los datos ingresados.
- Texto dinamico sin `innerHTML` (LL-001): bootstrap JSON con `tojson` e insercion DOM por `textContent`/`value`; metadata interna mostrada de solo lectura y re-estampada por el servidor al guardar.
- Si el CV no es estructurado valido (legacy, invalido, stale o version futura), el editor arranca desde los campos legacy con advertencia explicita; la conversion a estructurado ocurre solo al guardar.

- Rutas aditivas: `GET/POST /cvs/new/structured` y `GET/POST /cvs/{cv_id}/structured-edit`; boton `Crear CV estructurado` en el listado de CVs y `Editar estructurado` en el detalle. Los flujos plano y legacy siguen intactos.

Export/import JSON schema 2 con compatibilidad schema 1

- Un CV estructurado valido exporta un envelope schema 2 lossless con `schema_version`, `exported_at`, `source_cv_id` y `cv` con `title` y `structured_payload` completo; el filename usa la variante `cv-schema2`.
- CVs legacy, con payload invalido, stale o de version futura exportan schema 1 plano desde el snapshot legacy; nunca se mezclan payload y campos planos.
- El enlace `Descargar JSON plano (schema 1)` fuerza el export plano con query `schema_version=1`; el export schema `2` sobre un CV no estructurado se rechaza (400) y otros valores se rechazan sin downcast.
- El panel de importacion acepta JSON schema 1 (plano) y schema 2 (estructurado) con limite de `512 KB`: schema 2 crea un CV nuevo estructurado y schema 1 mantiene el flujo legacy; la importacion nunca sobrescribe el CV de origen.

TEX/PDF y ATS sobre representacion normalizada

- Las 4 plantillas (`Classic`, `Modern`, `Compact`, `Tech`) consumen una representacion normalizada unica: un CV estructurado valido usa el payload completo (incluye secciones solo-estructuradas) y cualquier otro caso usa el camino flat legacy, con TEX legacy byte-identico.
- El chequeo ATS mantiene los mismos criterios, pesos, topes y umbrales: el texto normalizado incluye el contenido solo-estructurado y excluye la metadata interna; no hay criterios nuevos ni IA.

Git Flow usado

1. `main` como rama estable.
2. `development` como rama de integracion.
3. `feature/*` por etapa o mini-etapa.
4. Merge fast-forward solo despues de validacion explicita.

Trazabilidad de prompts y comandos

- Los Prompt IDs son externos a Codex y sirven solo para trazabilidad entre Franco y ChatGPT.
- Los Prompt IDs no deben incluirse dentro del texto ejecutable que Codex recibe como orden.
- `COMMAND_LOG.md` registra comandos nuevos con timestamp local, etapa, ID secuencial, accion, motivo, comando, argumentos, resultado, error completo y reintento si aplica.
- Los documentos de desarrollo separan responsabilidades: `DEVELOPMENT_LOG.md` para hitos, `MVP_VALIDATION.md` para validaciones fuertes, `CHANGELOG_GENERAL.md` para releases y `MODULE_INDEX.md` para mapa de modulos.
- Si un dato historico no puede reconstruirse con certeza, se debe documentar `timestamp exacto no reconstruido con certeza`.

Regla de bitacora

Cada comando nuevo relevante debe poder leerse despues sin depender del historial de chat: que se hizo, por que, con que comando exacto y que resultado tuvo.

PRs y Codex Review

- Cada etapa integrada paso por rama feature dedicada.
- Se abrieron PRs hacia `development` para revision manual.
- Codex Review se uso como control de calidad antes de mergear.
- El PR `#8` ya fue mergeado previamente en `development` y dejo integrada la UI privada con ATS.
- El PR `#9` fue mergeado por squash hacia `development`, luego `main` fue sincronizada por fast-forward y el tag `v0.9.0` quedo publicado sobre `cbd10fa`.

Validaciones realizadas

- `python -m compileall app tests`
- `docker compose build`
- `docker compose up -d`
- `docker compose ps`
- `docker compose exec app python -m pytest`
- Validaciones HTTP y visuales sobre dashboard, modulos y exports
- Verificacion de extraccion PDF con `pdftotext`
- Validaciones de Feature 1: ver `docs/development/MVP_VALIDATION.md` (seccion T6)

Estado release v0.9.0

- Release actual publicado: `tag v0.9.0`
- Commit estable actual: `cbd10fa chore(release): prepare v0.9.0 (#9)`
- Release anterior: `tag v0.8.0`
- Commit de referencia anterior: `eab9556 fix(docs): render documentation as html with pdf downloads`
- Base integrada actual: `main` y `development` sincronizadas en `cbd10fa`.
- Dashboard privado disponible.
- Sidebar persistente disponible.
- Dark/light disponible.
- Flujo `Curriculum Vitae + ATS` disponible.
- Badge ATS en listado de CVs disponible.
- Modal ATS compacto disponible.
- Fix XSS de confirmacion aplicado.
- Documentacion interna alineada al release.
- El editor estructurado, el export JSON schema 2 y el render normalizado (Feature 1) no forman parte de este release: quedan sin tag en `feature/structured-cv-complete`.

Riesgos conocidos

- No hay login ni permisos.
- No hay CSRF ni hardening para despliegue remoto.
- SQLite no aplica claves foraneas duras entre modulos.
- La imagen Docker incluye dependencias de LaTeX y test, por lo que es pesada.

Backlog posterior

- Documentacion PDF futura adicional si hiciera falta.
- Login/auth si el alcance deja de ser local.
- Migraciones o mayor integridad relacional si la persistencia crece.
- Mejoras de export ATS y coverage end-to-end.
- Drag and drop.
- IA asistida.
- Miniaturas reales de PDF.
- Paletas visuales.

Fuera del alcance actual

Drag and drop, IA, miniaturas reales y paletas funcionales siguen como backlog. El editor estructurado queda implementado sin publicar (ver Feature 1 sin publicar). Ninguno de estos items forma parte de `v0.9.0`.

Historial y rollback

- Referencia operativa: `docs/development/PROJECT_HISTORY_ROLLBACK.md`
- El archivo documenta el tag estable actual `v0.9.0`, el release anterior `v0.8.0` y comandos de inspeccion o rollback temporal con advertencia de no ejecutarlos sin validacion previa.